ARTIFICIAL INTELLIGENCE

ECS601 • Professional Core Course-12

B.Tech (CSE) • Semester VI • Session 2024-25

✦ Complete Theory for Internal & External Exams ✦

✦ All 5 Units Covered with Memory Tricks ✦

✦ Exam Tips, Mnemonics & Visual Diagrams ✦

L-3 • T-1 • P-0 • C-4

TABLE OF CONTENTS

Your Complete Roadmap to AI

5 Hours

COURSE OUTCOMES (COs)

What you'll master by exam day

Know
Hours

CO1	**Understanding AI History & Algorithms** Know the history of AI, key milestones, classical algorithms like DFS, BFS, Hill Climbing.
CO2	**Applying AI Principles** Apply AI in problem solving, inference, perception, knowledge representation & learning.
CO3	**Prolog & Reasoning** Understand Prolog, Symbolic reasoning, Monotonic & Non-Monotonic reasoning.
CO4	**Knowledge Representation** Understand slot, filler techniques, Natural Language Processing concepts.
CO5	**Expert Systems** Understand Expert Systems, MYCIN & DENDRAL tools and their components.

UNIT 1

ARTIFICIAL INTELLIGENCE FUNDAMENTALS

Search • Problems • Heuristics • Constraints

8 Hours

MOVIE: 'The AI Robot Searches for Its Home'

Imagine a robot (AI) who must find its HOME by walking through a MAZE (State Space). It tries DFS (goes deep), BFS (level by level), Hill Climbing

1.1 WHAT IS ARTIFICIAL INTELLIGENCE?

Definition:

Artificial Intelligence (AI) is the branch of computer science concerned with making machines perform tasks that would normally require human intelligence, such as reasoning, learning, problem solving, perception, and language understanding.

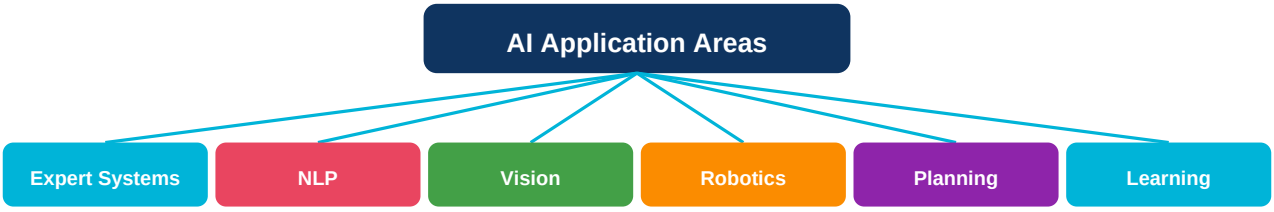
Researchers define AI in four ways — think of a 2×2 grid:

- **Thinking Humanly** — Cognitive modelling (mimics how humans think)
- **Acting Humanly** — Turing Test (behaves like a human)
- **Thinking Rationally** — Laws of thought (logic-based reasoning)
- **Acting Rationally** — Rational agent (makes best decisions)

Turing Test (Alan Turing, 1950):

A human interrogator communicates (via text) with an unknown entity. If the interrogator cannot reliably tell whether the entity is human or machine, the machine passes the Turing Test. This test evaluates intelligence through behavior — not through internal workings.

1.2 AREAS & IMPORTANCE OF AI



AI is important because it automates complex reasoning, handles uncertainty, learns from data, processes natural language, and enables autonomous systems. Key areas include expert systems, computer vision, natural language processing, robotics, machine learning, and planning.

1.3 HISTORY OF AI (Timeline for Exam)

Year	Event	Key Person
1943	McCulloch-Pitts neuron model	McCulloch & Pitts
1950	Turing Test proposed	Alan Turing

1956	AI term coined at Dartmouth	John McCarthy
1957	General Problem Solver	Newell & Simon
1965	DENDRAL expert system	Feigenbaum
1969	SHAKEY robot	Stanford RI
1972	PROLOG language	Colmerauer
1980	MYCIN expert system	Shortliffe
1997	Deep Blue beats Kasparov	IBM
2011	Watson wins Jeopardy!	IBM
2016	AlphaGo beats champion	DeepMind

1.4 PROBLEM SOLVING: STATE SPACE SEARCH

State: A description of the world at a specific point in time.

Initial State: The starting configuration of the problem.

Goal State: The desired final configuration.

Operators: Actions/moves that change one state to another.

State Space: The set of all possible states reachable from the initial state.

Path: A sequence of states connected by operators.

Solution: A path from initial state to goal state.

Problem Formulation (4 essential components):

Production System (for Theory Questions):

A Production System consists of: (1) **Global Database** — stores current problem state; (2) **Production Rules** — condition-action pairs (IF condition THEN action); (3) **Control Strategy** — decides which rule to apply when multiple rules match.

Properties of a good production system: Completeness, Consistency, Modularity, Efficiency.

1.5 BLIND SEARCH: DFS & BFS

DFS = 'Dig Deep First!' | BFS = 'Broad Sweep First!'

DFS is like digging a well — go as deep as possible before trying another spot. BFS is like a flood — spread equally in all directions level by level.

Property	DFS (Depth-First)	BFS (Breadth-First)
Data Structure	Stack (LIFO)	Queue (FIFO)
Memory	O(depth×b)	O(b^d)

Time	O(b^m)	O(b^d)
Completeness	No (may loop)	Yes
Optimality	No	Yes (unit cost)
Best When	Deep solutions, low memory	Shallow solutions, completeness needed

DFS Algorithm (must know for exam):

```
1. Push initial state onto STACK
2. WHILE stack not empty DO:
a. Pop state from stack (call it N)
b. IF N is goal THEN return SUCCESS
c. ELSE expand N, push all children onto stack
3. IF stack empty THEN return FAILURE
```

BFS Algorithm:

Same as DFS but use QUEUE instead of STACK (enqueue children, dequeue front node each step).

1.6 HEURISTIC SEARCH

A **heuristic** is an estimate (rule of thumb) that guides search toward the goal. It is domain-specific knowledge that helps reduce the search space. Denoted as h(n) = estimated cost from node n to goal.

- **Admissible heuristic**: Never overestimates actual cost. h(n) ≤ h*(n)
- **Consistent heuristic**: h(n) ≤ cost(n,n') + h(n') for all neighbors n'

Hill Climbing:

Like climbing a hill in fog — always move to the neighbor with the highest value (closest to goal). Greedy approach: never backtrack.

Types of Hill Climbing:

- **Simple Hill Climbing**: Move to FIRST better neighbor found.
- **Steepest Ascent Hill Climbing**: Evaluate ALL neighbors, move to BEST one.
- **Stochastic Hill Climbing**: Randomly pick among uphill neighbors.

- **Local Maximum**: Stuck at a peak that is not the global peak. No neighbor is better but goal not reached.
- **Plateau**: A flat region where all neighbors have same value. No direction to move.
- **Ridge**: A slope that climbs in a direction the search cannot take. Appears as barrier.

Solution: Random restarts, Simulated Annealing, or switch to Best First Search.

Best First Search (BFS with heuristic):

Uses a priority queue (min-heap). Always expands the node with the LOWEST h(n) value (best estimated cost to goal). More informed than Hill Climbing because it can backtrack.

A* Search (A-Star) — Most Important Heuristic Algorithm:

- **g(n)** = Actual cost from start to node n (known)
- **h(n)** = Estimated cost from n to goal (heuristic)
- **f(n)** = Total estimated cost of path through n

A* is COMPLETE and OPTIMAL when h(n) is admissible. It expands least f(n) node first.

Generate and Test:

Simple two-step method: (1) Generate a possible solution; (2) Test if it satisfies the goal. If not, generate another. Used in DENDRAL. Effective for small solution spaces.

1.7 PROBLEM REDUCTION (AO* Algorithm)

Some problems are best solved by decomposing them into sub-problems. This is called **Problem Reduction**. An AND-OR graph is used: OR nodes mean choose one alternative; AND nodes mean solve ALL sub-problems.

AO* Algorithm solves AND-OR graphs optimally. It finds the minimum cost solution graph, where AND nodes require solving all children and OR nodes require solving the best child.

1.8 CONSTRAINT SATISFACTION PROBLEMS (CSP)

- **Variables**: X1, X2, ..., Xn (what we need to assign)
- **Domains**: D1, D2, ..., Dn (possible values for each variable)
- **Constraints**: Rules that limit variable assignments
- **Solution**: Assignment of values to all variables satisfying ALL constraints

Example: Map Coloring — Variables=regions, Domain={R,G,B}, Constraint=adjacent regions different color.

CSP Solving Techniques:

- **Backtracking**: Assign values one by one, backtrack when constraint violated.
- **Forward Checking**: After each assignment, eliminate illegal values from unassigned variables.
- **Arc Consistency (AC-3)**: Ensure every value in a domain has valid support.

1.9 CRYPT ARITHMETIC

Crypt Arithmetic is a type of mathematical puzzle where digits are represented by letters. The goal is to find a digit assignment such that the arithmetic equation holds.

Each letter represents a unique digit (0-9). Leading digits cannot be 0.

Solution: S=9, E=5, N=6, D=7, M=1, O=0, R=8, Y=2

```
9567 + 1085 = 10652 ✓
```

Strategy: Use constraint propagation + backtracking.

★ EXAM TIPS FOR UNIT 1:

- Draw DFS and BFS trees step-by-step — examiners love this!
- Know A* formula f(n)=g(n)+h(n) and at least one worked example
- For Hill Climbing — always mention the 3 problems: Local Max, Plateau, Ridge
- CSP: mention backtracking + constraint propagation
- Production System has 3 components: Global DB + Rules + Control Strategy

UNIT 2

KNOWLEDGE REPRESENTATION

Logic • Inference • Chaining • Resolution

8 Hours

MOVIE: 'The Detective's Knowledge Map'

Imagine Sherlock Holmes (AI) building a MAP of FACTS (logic) and using DEDUCTION (resolution) to solve cases. He uses Forward Chaining

2.1 KNOWLEDGE REPRESENTATION: INTRO

Knowledge Representation (KR) is the area of AI concerned with how knowledge about the world can be represented and used by AI systems to solve complex problems.

Properties of good KR scheme:

- **Representational Adequacy**: Can represent all needed knowledge
- **Inferential Adequacy**: Can derive new knowledge from existing
- **Inferential Efficiency**: Derives knowledge efficiently
- **Acquisitional Efficiency**: Easy to add new knowledge

Types of Knowledge:

Declarative | Procedural | Meta | Heuristic | Structural

• **Declarative**: Facts about the world (what is true). E.g., 'All birds can fly' • **Procedural**: How to do things (algorithms, rules). E.g., 'To multiply: ...' • **Meta-knowledge**: Knowledge about knowledge (what we know we know) • **Heuristic**: Rules of thumb based on experience

2.2 PROPOSITIONAL LOGIC

Propositional Logic (PL) uses propositions (statements that are TRUE or FALSE) connected by logical connectives.

Connective	Symbol	Meaning	Example
NOT	¬ (or ~)	Negation	¬P = 'Not P'
AND	∧	Conjunction	P ∧ Q = 'P and Q'
OR	∨	Disjunction	P ∨ Q = 'P or Q'
IMPLIES	→	Implication	P → Q = 'If P then Q'
BICONDITIONAL	↔	Equivalence	P ↔ Q = 'P iff Q'

2.3 PREDICATE LOGIC (First Order Logic - FOL)

Predicate Logic extends Propositional Logic with **variables, predicates, functions,** and **quantifiers**. It can represent relationships between objects.

- **Constants**: Specific objects — John, Mary, 5
- **Variables**: x, y, z — range over objects
- **Predicates**: Properties or relations — Loves(x,y), Human(x)
- **Functions**: Return values — FatherOf(x), Plus(x,y)
- **Quantifiers**: ∀ (for all) and ∃ (there exists)

Example: 'All humans are mortal' → ∀x: Human(x) → Mortal(x)

Example: 'Socrates is human' → Human(Socrates)

Therefore: Mortal(Socrates) — derived by inference!

2.4 RESOLUTION (Theorem Proving)

Resolution is a rule of inference used to prove theorems by contradiction. Given two clauses that contain complementary literals, resolution produces a new clause.

If Clause 1 = (A ∨ B) and Clause 2 = (¬B ∨ C)

Then Resolvent = (A ∨ C) — the B and ¬B cancel out!

Proof by Refutation: To prove P, negate it (assume ¬P), add to knowledge base, and derive contradiction (empty clause ■). If empty clause derived → original P is TRUE.

Steps for Resolution Proof:

1. Convert all statements to **Conjunctive Normal Form (CNF)** — AND of OR clauses
2. **Negate** the goal/query
3. Add negated goal to clause set
4. Apply resolution rule repeatedly
5. If empty clause (■) derived → PROVED!

Unification:

Unification is the process of finding substitutions for variables so that two expressions become identical. Example: Unify Loves(x, Mary) with Loves(John, y) → {x/John, y/Mary}.

2.5 FORWARD & BACKWARD CHAINING

■

CHAIN MEMORY TRICK

FORWARD = 'From Evidence to Crime' (detective starts with clues, reaches conclusion). BACKWARD = 'Reverse from Crime to Evidence' (starts with goal,

Property	Forward Chaining	Backward Chaining
Direction	Data → Goal (bottom-up)	Goal → Data (top-down)
Driven by	Available facts/data	Goal/query to prove
Also called	Forward reasoning	Backward reasoning / Goal-directed
Use case	Monitoring, alerting	Diagnosis, query answering
Used in	CLIPS, OPS5	Prolog, MYCIN

Efficiency	May derive useless facts	Only derives relevant facts

Matching & Control Knowledge:

Matching is the process of comparing patterns (rule LHS) with the current state in the knowledge base to find applicable rules. **Control Knowledge** (meta-knowledge) guides which rule to fire when multiple match — this is the Conflict Resolution Strategy.

Conflict Resolution Strategies:

- **Recency**: Prefer rules matching most recently added facts
- **Specificity**: Prefer more specific rules over general ones
- **Priority**: Assign priorities to rules
- **Complexity**: Prefer rules with more conditions matched

★ EXAM TIPS FOR UNIT 2:

- Write FOL examples: ∀x Human(x)→Mortal(x) is CLASSIC — always include it
- Resolution: Show CNF conversion → negate goal → resolve → empty clause
- Forward vs Backward Chaining: Draw a simple example tree for each
- Unification: Show a simple substitution example
- Remember: Declarative = facts, Procedural = how-to

UNIT 3

AI PROGRAMMING: PROLOG

Facts • Rules • Variables • Structures • Reasoning

8 Hours

SONG: 'Prolog Facts, Rules, Variables — FRV!'

Sing: 'FACTS are true things, RULES say IF-THEN, VARIABLES are X and Y!
PROLOG thinks backwards, it BACKTRACKS to find WHY!' — Remember: FRV =

3.1 INTRODUCTION TO PROLOG

Prolog (PROgramming in LOGic) was developed by Alain Colmerauer in 1972. It is a declarative language based on first-order predicate logic. Instead of telling the computer HOW to solve a problem, you tell it WHAT is true (facts and rules), and Prolog figures out the solution.

- **Declarative**: Describe relationships, not procedures
- **Backtracking**: Automatically backtracks to find all solutions
- **Unification**: Pattern matching built-in
- **Recursion**: Primary control structure
- **Logic-based**: Based on Horn clauses (subset of FOL)

3.2 PROLOG BASICS: FACTS, RULES, VARIABLES

FACTS: Unconditionally true statements (always end with period)

```
likes(mary, food). % Mary likes food
likes(mary, wine). % Mary likes wine
likes(john, wine). % John likes wine
likes(john, mary). % John likes Mary
```

RULES: Conditional statements — IF body THEN head

```
friends(X,Y) :- likes(X,Y), likes(Y,X). % X and Y are friends if they like each other
happy(X) :- likes(X, food). % X is happy if X likes food
```

VARIABLES: Start with UPPERCASE or underscore

```
?- likes(mary, X). % Query: What does Mary like?
X = food ; % Answer 1: food
X = wine. % Answer 2: wine
```

- **Atoms**: Start with lowercase — john, food, mary
- **Variables**: Start with uppercase or _ — X, Y, Name, _temp
- **Structures**: functor(arg1, arg2) — likes(john, mary)
- **:-** means 'IF' (neck symbol)
- **%** starts a comment
- Every clause ends with **.** (period)
- **?-** starts a query

3.3 DATA STRUCTURES: LISTS AND TREES

Lists in Prolog:

Lists are the primary data structure. Format: [Head|Tail]

```
[1,2,3,4] → Head=1, Tail=[2,3,4]
[H|T] = [a,b,c] → H=a, T=[b,c]
[] is the empty list
```

Common List Operations:

```
member(X,[X|_]). % X is member of list starting with X
member(X,[_|T]) :- member(X,T). % Or X is member of tail T
append([],L,L). % Append empty list to L gives L
append([H|T],L,[H|R]) :- append(T,L,R). % Append rule
```

Trees in Prolog:

Trees are represented using structures. A binary tree with root R, left L, right R: tree(R, L, R). Empty tree: nil.

```
tree(node(8, node(3,nil,nil), node(10,nil,nil)))
```

3.4 CUT (!) IN PROLOG

The **cut** (!) operator in Prolog stops backtracking. Once the cut is reached, Prolog commits to the current choice and will not try alternatives for the parent goal.

```
max(X,Y,X) :- X >= Y, !. % If X >= Y, X is max, no backtrack
max(_,Y,Y). % Otherwise Y is max
```

- **Green cut**: Does not change meaning, only efficiency (removes redundant backtracking)
- **Red cut**: Changes meaning — if removed, program behaves differently
- Implementing if-then-else, negation as failure

3.5 SYMBOLIC REASONING

Symbolic Reasoning is reasoning using symbols and rules rather than numerical computation. Prolog is inherently a symbolic reasoning system — it manipulates symbols (atoms, variables, structures) according to logical rules.

3.6 MONOTONIC vs NON-MONOTONIC REASONING

Property	Monotonic Reasoning	Non-Monotonic Reasoning
Behavior	Once concluded, always true	Conclusions can be retracted
New info effect	Only adds new conclusions	Can remove old conclusions
Example	Mathematics, formal logic	Default reasoning, belief revision
Prolog default	Yes (monotonic)	Can be done with CUT/assert
Real world	Not always suitable	More realistic/flexible

Reasoning Under Uncertainty:

In the real world, knowledge is often incomplete or uncertain. Methods include: **Probability Theory** (Bayesian), **Fuzzy Logic** (degrees of truth), **Certainty Factors** (used in MYCIN), and **Dempster-Shafer Theory**.

3.7 DFS & BFS IMPLEMENTATION IN PROLOG

DFS in Prolog (natural — Prolog uses DFS by default!):

```
solve(Goal) :- call(Goal). % Base case: goal is true
dfs(Node, Goal, Path) :- % DFS template
Node = Goal, Path = [Goal]. % Found goal
dfs(Node, Goal, [Node|Path]) :- % Not goal
arc(Node, Next), % Move to neighbor
dfs(Next, Goal, Path). % Recurse
```

BFS in Prolog requires explicit queue management using lists, since Prolog's natural search is DFS.

★ EXAM TIPS FOR UNIT 3:

- Prolog FACTS end with period. RULES use :- (read as 'if')
- Variables = Uppercase. Atoms = lowercase
- List notation [H|T] is very important — write member and append predicates
- Cut (!) prevents backtracking — know green vs red cut
- Prolog IS monotonic by default; non-monotonic needs assert/retract
- DFS is Prolog's default search strategy!

UNIT 4

SLOT & FILLER STRUCTURES + NLP

Semantic Nets • Frames • Scripts • Language Processing

8 Hours

MOVIE: 'The Language Professor's Framework'

A Professor (AI) organizes knowledge using NETS (webs of concepts), FRAMES (filing cabinets with slots), and SCRIPTS (pre-planned scenarios like 'going

4.1 SEMANTIC NETWORKS

A **Semantic Network** (Semantic Net) is a graph-based KR scheme where nodes represent objects/concepts and edges represent relationships between them.

- **Nodes**: Objects, concepts, events (e.g., Animal, Bird, Tweety)
- **Links/Edges**: Relationships between nodes
- **ISA link**: Inheritance (Canary IS-A Bird IS-A Animal)
- **AKO link**: A-Kind-Of (subset relationship)
- **HAS-A link**: Possession (Bird HAS-A Wings)
- **Property inheritance**: Lower nodes inherit properties of higher nodes

Example: Tweety IS-A Canary IS-A Bird IS-A Animal. Tweety inherits: has wings, can fly (unless overridden)

Advantages of Semantic Nets: Natural representation, easy visualization, property inheritance. **Disadvantages:** No clear semantics, difficult to handle negation and quantifiers.

4.2 FRAMES

A **Frame** is a data structure for representing stereotyped situations. Introduced by Marvin Minsky (1975). Like an object in OOP — has **slots** (attributes) and **fillers** (values).

FRAME: PERSON	
Slot	**Filler / Default**
Name	? (must fill)
Age	? (must fill)
Gender	?
Occupation	Unknown (default)
Address	Unknown (default)
ISA	Living-Being

Slot Types in Frames:

- **Value slots**: Hold actual values
- **Default slots**: Hold default values (used when no value given)
- **Procedural slots (Demons)**: IF-ADDED, IF-NEEDED, IF-REMOVED — trigger actions

• Frames support **inheritance** through ISA links between frames

4.3 CONCEPTUAL DEPENDENCY (CD)

Developed by Roger Schank. CD theory represents the meaning of sentences in a language-independent way using 11 primitive actions (ACTs).

ATRANS (transfer ownership), PTRANS (physical transfer/move), PROPEL (apply force), MOVE (body part), GRASP (grasp object), INGEST (eat/drink), EXPEL (emit), MTRANS (transfer mental info), MBUILD (create mental concept), CONC (think about), SPEAK (produce sound)

Example: 'John ate the pizza' → INGEST(John, pizza, mouth)

4.4 SCRIPTS

A **Script** is a structured representation of a stereotyped sequence of events for a familiar situation. Like a Frame for dynamic situations (scenarios) rather than static objects.

Track: Coffee shop / Fine dining

Props: Tables, menu, food, money, bill

Roles: Customer, waiter, chef, cashier

Entry conditions: Customer is hungry, has money

Scenes:

Scene 1 (Entering): Customer enters, finds table, sits
Scene 2 (Ordering): Reads menu, calls waiter, orders food
Scene 3 (Eating): Food arrives, customer eats
Scene 4 (Exiting): Gets bill, pays, leaves tip, exits

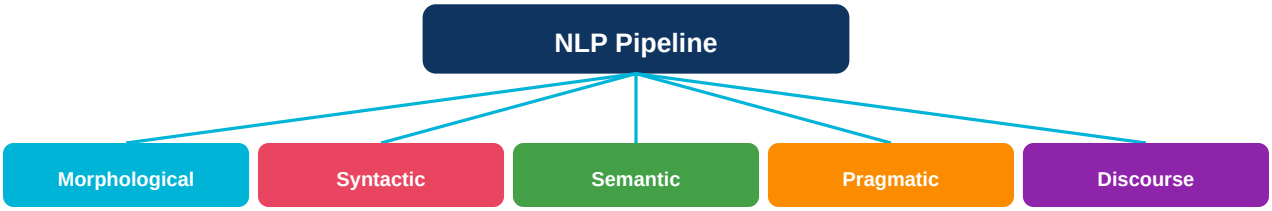
Results: Customer not hungry, restaurant has money

4.5 CYC PROJECT

The **CYC project** (started by Doug Lenat in 1984) aims to build a massive common-sense knowledge base. CYC contains millions of facts and rules about everyday life that humans take for granted. It uses a formal language (CycL) based on predicate logic.

4.6 NATURAL LANGUAGE PROCESSING (NLP)

NLP enables computers to understand, interpret, and generate human language. It bridges the gap between human communication and computer understanding.



Level	Focus	Example

Morphological	Word structure, affixes	'running' = run + -ing
Syntactic	Grammar, sentence structure	Parsing 'The cat sat'
Semantic	Meaning of words & sentences	Word sense disambiguation
Pragmatic	Context, speaker intent	Interpreting 'Can you pass salt?'
Discourse	Multi-sentence coherence	Pronoun resolution

4.7 PARSING TECHNIQUES

Syntactic Parsing:

Parsing = analyzing sentence structure according to grammar rules. A **Context-Free Grammar (CFG)** defines rules like: S → NP VP, NP → Det N, VP → V NP.

Types of Parsers:

- **Top-Down Parser**: Starts from S (start symbol), expands until reaching words
- **Bottom-Up Parser**: Starts from words, reduces to S
- **Chart Parser (Earley)**: Efficient, handles ambiguity
- **CYK Algorithm**: Dynamic programming parser for CNF grammars

4.8 AUGMENTED TRANSITION NETWORKS (ATN)

An **ATN** is a finite automaton extended with: recursion (to handle embedded sentences), registers (to store parsed components), and tests/actions on transitions. ATNs are used for parsing complex natural language sentences.

- **States**: Represent parsing progress
- **Arcs**: Transitions labeled with word categories or non-terminals
- **Registers**: Store parsed phrases (SUBJ, OBJ, etc.)
- **Actions**: SETR (set register), SENDR (send to lower network)

Example arc: (S/1) → CAT NP → (S/2) means: from state S/1, parse a NP, go to S/2

Semantic Analysis — Case Grammar:

Case grammar (Fillmore, 1968) represents the semantic roles of noun phrases: Agent (doer), Patient (receiver), Instrument (tool), Location, Time, Beneficiary. These semantic roles are language-independent.

Example: 'John cut the bread with a knife' → Agent=John, Patient=bread, Instrument=knife

★ EXAM TIPS FOR UNIT 4:

- Draw a Semantic Network with ISA and HAS-A links — very popular question
- Frame example: Draw BIRD frame with slots (has-wings, can-fly, etc.)
- Script: Write the RESTAURANT script with all 4 scenes
- NLP levels: Remember the 5 levels — Morphological→Syntactic→Semantic→Pragmatic→Discourse
- ATN: Know the 4 components (states, arcs, registers, actions)
- CD theory: 11 primitive ACTs — at least mention ATRANS, PTRANS, MTRANS, INGEST

UNIT 5

EXPERT SYSTEMS

Knowledge Engineering • MYCIN • DENDRAL • Shells

8 Hours

MOVIE: 'The Digital Doctor — MYCIN'

Imagine a DIGITAL DOCTOR (Expert System) who has the KNOWLEDGE of 100 specialists stored in his KNOWLEDGE BASE. Patients ask questions, the

5.1 WHAT IS AN EXPERT SYSTEM?

An **Expert System (ES)** is a computer program that emulates the decision-making ability of a human expert in a specific domain. It uses a Knowledge Base of expert knowledge and an Inference Engine to derive conclusions.

> ***"An expert system is an intelligent computer program that uses knowledge and inference procedures to solve problems that are difficult enough to require significant human expertise for their solution." — Edward Feigenbaum***

5.2 CHARACTERISTICS OF EXPERT SYSTEMS

Characteristic	Description
High Performance	Performs at the level of a human expert
Reliability	Produces consistent results every time
Understandability	Can explain its reasoning (unlike black-box ML)
Flexibility	Can work with incomplete or uncertain information
Domain-specific	Deep knowledge in ONE specific domain
Separation	Knowledge and reasoning separated (KB + IE)
Transparency	Provides explanation and justification
Graceful degradation	Still works reasonably with incomplete data

5.3 ARCHITECTURE OF EXPERT SYSTEMS

ARCHITECTURE MNEMONIC: 'KB IE WM UI EF'

Know It Well — Use It Effectively! KB=Knowledge Base, IE=Inference Engine, WM=Working Memory, UI=User Interface, EF=Explanation Facility. These 5 are

Expert System Architecture Components:

• **Knowledge Base (KB)**: Contains domain-specific facts and rules (IF-THEN rules). The heart of the ES. Maintained by Knowledge Engineer.

• **Inference Engine (IE)**: The 'brain' that applies rules to facts to derive conclusions. Uses Forward Chaining or Backward Chaining.

• **Working Memory (WM)**: Temporary storage for current problem facts (user inputs, intermediate conclusions).

• **User Interface**: Allows non-expert users to communicate with the system (question-answer dialogue).

• **Explanation Facility**: Explains HOW a conclusion was reached (WHY questions) and WHY a question is being asked. Critical for user trust.

• **Knowledge Acquisition Facility**: Helps knowledge engineers add/modify knowledge in the KB.

5.4 KNOWLEDGE ENGINEERING

Knowledge Engineering is the process of acquiring, encoding, and maintaining knowledge in an expert system. The **Knowledge Engineer** (KE) is the bridge between the human expert and the ES.

Knowledge Acquisition Methods:

- **Interviews**: Direct questioning of domain expert
- **Protocol Analysis**: Expert thinks aloud while solving problems
- **Observation**: Watch expert solve real cases
- **Document Analysis**: Extract knowledge from books/manuals
- **Induction**: Learn rules from examples (machine learning)
- **Repertory Grids**: Structured interview technique

> The most challenging aspect of building ES is getting knowledge OUT of the expert's head and INTO the computer. This is called the **Knowledge Acquisition Bottleneck**. Experts find it hard to articulate their implicit knowledge.

5.5 EXPERT SYSTEM LIFE CYCLE

Life Cycle Phases:

1. **Problem Identification**: Define scope, check if ES is suitable
2. **Feasibility Study**: Is the domain suitable? Is an expert available?
3. **Knowledge Acquisition**: Extract knowledge from expert
4. **Design**: Choose shell, design KB structure, define inference strategy
5. **Testing & Validation**: Test on sample cases, verify against expert
6. **Deployment**: Install at user site, train users
7. **Maintenance**: Update KB with new knowledge, fix errors

5.6 EXPERT SYSTEM SHELLS

An **Expert System Shell** is an ES with the domain knowledge removed — it provides the inference engine and user interface, but the developer adds the domain-specific KB. Shells make building ES faster.

Shell Name	Inference	Language	Features
CLIPS	Forward Chaining	C	Fast, free, widely used

JESS	Forward Chaining	Java	Java-based CLIPS clone
OPS5	Forward Chaining	Lisp	Early landmark shell
Prolog	Backward Chaining	Prolog	Logic-based, built-in search
MYCIN (base)	Backward Chaining	Lisp	Used certainty factors
EMYCIN	Backward Chaining	Lisp	Empty MYCIN shell
ART	Both	Lisp	Advanced Reasoning Tool

5.7 MYCIN — MEDICAL EXPERT SYSTEM

MYCIN = 'MYcin Cures INfections!'

MYCIN was the landmark medical ES built at Stanford (1970s) by Edward Shortliffe. It diagnosed bacterial blood infections and recommended

- **Domain**: Bacterial infections of the blood and meningitis
- **Developer**: Edward Shortliffe, Stanford University, mid-1970s
- **Language**: LISP
- **Inference**: Backward Chaining (goal-directed)
- **Uncertainty**: Uses Certainty Factors (CF) from -1 to +1
- **Rules**: ~450 IF-THEN rules
- **Performance**: Equaled or exceeded human expert performance in tests
- **WHY/HOW**: Could explain its reasoning
- **EMYCIN**: Empty MYCIN shell — remove medical KB, add new domain
- **Not used in practice**: Legal/liability issues and lack of integration with patient records

Certainty Factors (CF) in MYCIN:

CF = MB - MD, where MB = Measure of Belief, MD = Measure of Disbelief. Range: -1 to +1.

- CF = +1: Certainly TRUE
- CF = -1: Certainly FALSE
- CF = 0: Unknown/no evidence either way
- CF > 0.2: Sufficient evidence to consider diagnosis

CF Combination Rules:

- IF (A CF x) AND (B CF y) THEN conclusion CF = min(x,y)
- IF (A CF x) OR (B CF y) THEN conclusion CF = max(x,y)
- Sequential rules: CF(total) = CF1 + CF2×(1 - CF1) when both positive

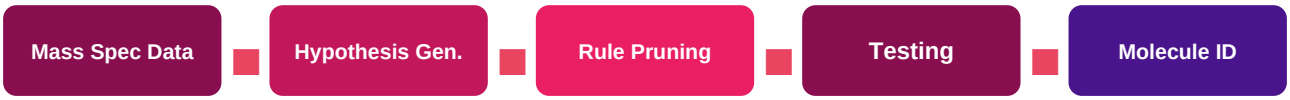
Sample MYCIN Rule:

```
RULE 050:
IF: 1) The infection is primary bacteremia, AND
2) The site of the culture is one of: blood
3) The suspected portal of entry is GI tract
THEN: Bacteroides fragilis is an organism [CF = 0.7]
```

5.8 DENDRAL — CHEMISTRY EXPERT SYSTEM

- **Domain**: Identifying molecular structures from mass spectrometry data
- **Developers**: Edward Feigenbaum, Joshua Lederberg, Bruce Buchanan — Stanford, 1965
- **First successful AI expert system** in history
- **Language**: LISP
- **Approach**: Generate and Test + Domain Heuristics
- **Performance**: Could identify molecular structures as well as PhD chemists
- **Meta-DENDRAL**: Extension that could LEARN new rules from data (early ML!)
- **Significance**: Proved that computers could match human expert performance in a real domain

DENDRAL Process:



Feature	MYCIN	DENDRAL
Domain	Blood infections	Molecular structure
Year	1970s	1965
Inference	Backward Chaining	Generate & Test
Uncertainty	Certainty Factors	Domain heuristics
Field	Medicine	Chemistry
Knowledge	~450 rules	Structural chemistry rules
Significance	Medical AI pioneer	First successful ES
Shell	EMYCIN (derived)	No standard shell

5.9 LIMITATIONS OF EXPERT SYSTEMS

- Cannot learn from experience (no ML in traditional ES)
- Knowledge acquisition is slow, expensive, difficult
- Brittle at boundaries of domain (no common sense)
- Maintenance is costly — outdated KB needs continuous updating
- Cannot handle novel situations outside KB
- Hard to represent procedural/temporal knowledge

★ EXAM TIPS FOR UNIT 5:

- Draw the Expert System Architecture diagram — 6 components!
- MYCIN: Domain=blood infections, CF range=-1 to +1, ~450 rules, Backward Chaining
- DENDRAL: First ES ever, chemistry, Generate & Test, 1965
- Knowledge Acquisition Bottleneck — very likely theory question
- EMYCIN = Empty MYCIN = Expert System Shell
- CF combination: AND=min, OR=max — easy marks!
- Characteristics: High Performance, Reliability, Understandability, Domain-specific

QUICK REVISION — IMPORTANT Q&A

Last night before exam — Read this!

★ Hours

Q1: What is Artificial Intelligence?

➤ AI is the simulation of human intelligence in machines to perform tasks like reasoning, learning, perception, and language understanding.

Q2: What is the Turing Test?

➤ A test by Alan Turing (1950) where a human interrogator cannot distinguish machine responses from human responses via text, proving machine intelligence.

Q3: Difference between DFS and BFS?

➤ DFS uses Stack, goes deep first, not optimal, O(b^m) time. BFS uses Queue, level by level, complete & optimal, O(b^d) time.

Q4: What is a Heuristic? What is A*?

➤ A heuristic h(n) estimates cost to goal. A* uses f(n)=g(n)+h(n) where g=actual cost, h=heuristic. A* is optimal if h is admissible.

Q5: What is a Production System?

➤ A system with 3 components: Global Database, Production Rules (IF-THEN), and Control Strategy. Used to represent knowledge and solve problems.

Q6: What is Resolution?

➤ A rule of inference: from (A∨B) and (¬B∨C), derive (A∨C). Used in theorem proving by contradiction (refutation).

Q7: Forward vs Backward Chaining?

➤ Forward: data→goal (bottom-up, fact-driven). Backward: goal→data (top-down, goal-directed, used in Prolog and MYCIN).

Q8: What are Facts, Rules, Variables in Prolog?

➤ Facts: unconditional statements (likes(john,mary).). Rules: conditional (happy(X):-likes(X,food).). Variables: uppercase (X, Y).

Q9: What is a Semantic Network?

➤ Graph KR scheme with nodes (objects) and edges (relationships like ISA, HAS-A). Supports property inheritance.

Q10: What is a Frame?

➤ Data structure for stereotyped situations with slots (attributes) and fillers (values). Supports defaults and procedural attachments.

Q11: What is a Script?

➤ Stereotyped sequence of events (like restaurant script). Has roles, props, entry conditions, scenes, and results.

Q12: What is an Expert System?

➤ Computer program that emulates human expert decision-making using a Knowledge Base and Inference Engine.

Q13: What are the components of an Expert System?

➤ Knowledge Base, Inference Engine, Working Memory, User Interface, Explanation Facility, Knowledge Acquisition Facility.

Q14: What is MYCIN?

➤ Medical ES (Stanford, 1970s) for diagnosing bacterial blood infections using ~450 rules, backward chaining, and certainty factors (CF: -1 to +1).

Q15: What is DENDRAL?

➤ First successful ES (Stanford, 1965) for identifying molecular structures from mass spectrometry data using Generate & Test approach.

Q16: What is a CSP?

➤ Problem with Variables, Domains, and Constraints. Solved using backtracking, forward checking, and arc consistency.

Q17: What is Crypt Arithmetic?

➤ Mathematical puzzle where letters represent digits. SEND+MORE=MONEY is the classic example. Solved by constraint propagation + backtracking.

Q18: What is Monotonic vs Non-Monotonic Reasoning?

➤ Monotonic: adding new info only adds conclusions (never removes). Non-Monotonic: new info can invalidate old conclusions (default reasoning).

Q19: What is NLP? Name its 5 levels.

➤ NLP enables computers to process human language. Levels: Morphological, Syntactic, Semantic, Pragmatic, Discourse.

Q20: What is an ATN?

➤ Augmented Transition Network: finite automaton with recursion, registers, and actions for parsing natural language sentences.

REFERENCES & RESOURCES

Books, Links, and Final Tips

■ Hours

TEXT BOOK:

Rich, E. and Knight, K., *Artificial Intelligence*, Tata McGraw Hill. (Latest edition recommended)

REFERENCE BOOKS:

1. Cloksin, W.F., Mellish, C.S., *Programming In Prolog*, Narosa Publishing House.
2. Janakiraman, V.S., Sarukesi, K., *Foundation of Artificial Intelligence & Expert System*, Macmillan.

ONLINE RESOURCES:

- NPTEL: https://nptel.ac.in/courses/106/105/106105077/
- YouTube: https://www.youtube.com/watch?v=JMUxmLyrhSk

For Internal Exams (Unit tests):

- Focus on definitions and algorithms with examples
- Draw diagrams — DFS/BFS trees, Semantic nets, ES architecture
- Always show step-by-step working for search algorithms

For External Exams:

- 2-mark questions: Give crisp definitions with one example
- 5-mark questions: Definition + characteristics/types + diagram
- 10-mark questions: Full theory + comparison table + diagram + example

Must-Know Diagrams:

DFS/BFS tree, A* example, Semantic Network, Frame structure, ES Architecture, MYCIN rule, Script scenario

Best of Luck! You've got this! ■

ECS601 — Artificial Intelligence — B.Tech CSE Session 2024-25